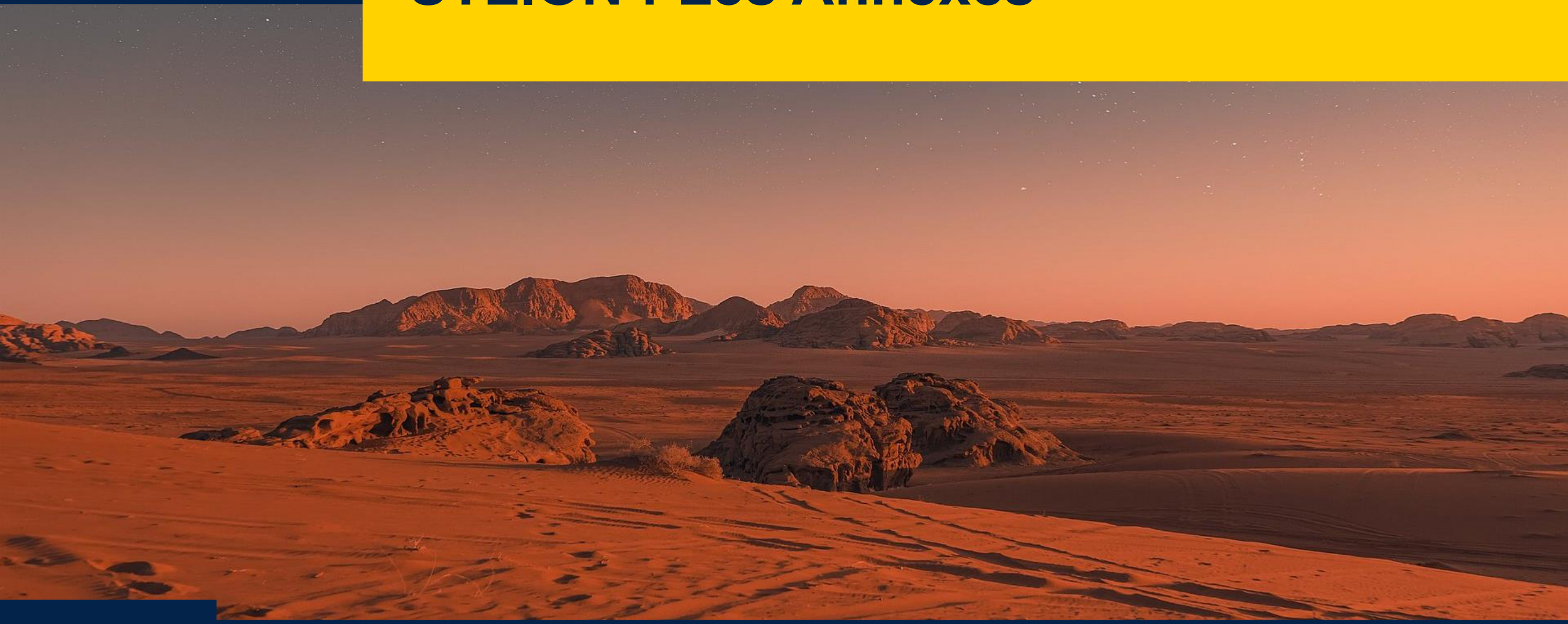
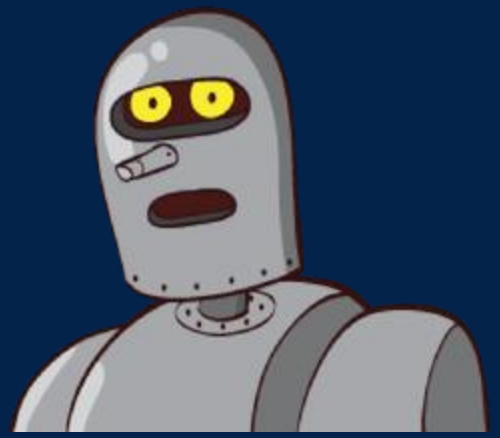
STLION : Les Annexes



life.augmented

Epreuve 3 – Programmation d’un simulateur



life.augmented

Quelques réponses à vos questions…

- **Que doit faire le simulateur ?**
  - Le simulateur doit permettre de prévoir le comportement du robot STLION quand il recevra un fichier d'instructions en binaire.
  - Le langage est libre, donc n'hésitez pas à vous faire plaisir et à proposer un simulateur original !
- **Que doit produire le simulateur au minimum ?**
  - Un rapport d'exécution comprenant pour chaque instruction :
    - L'instruction Assembleur décodée (OUT, PUSH, POP, …)
    - La valeur du paramètre le cas échéant (Rx, valeur)
    - Le coût en cycle de l'instruction
    - L'état des registres et de la mémoire APRES l'instruction
    - La position des pointeurs
  - Le rapport doit se finir avec un résumé des outputs et le coût total en cycles.

Epreuve 4 – Programmation de votre robot pour interpréter du binaire

Quelques conseils

- Vos travaux sur le simulateur doivent vous aider à réaliser cette partie
- Pensez que le micropython ne dispose pas des mêmes librairies que le python
- Même si STLION n’est pas entièrement fonctionnel à cause de son atterrissage « artistique », votre robot l’est. N’hésitez pas à exploiter les périphériques accessibles pour proposer quelques petites choses originales !
- Les moteurs de votre robot peuvent nécessiter une adaptation de votre code pour assurer sa capacité à avancer de façon fiable.
- Il faut coder et tester la communication avec l'envoi du binaire entre votre robot et votre PC via le dongle USB.
- N’oubliez pas d’identifier votre robot et carte supplémentaire pour la connexion BLE.

Démarrer votre robot

life.augmented

Installation du matériel et de son logiciel pour le piloter

- Le **robot** est livré « monté ». Vous pouvez le connecter à votre PC grâce au **câble USB fourni**.
- Les **3 cartes STM32WB55 de votre kit** (robot, dongle USB, carte supplémentaire) contiennent:
  - Un firmware pour l'interpréteur MicroPython: celui-ci est déjà présent, vous n'avez pas besoin d'y toucher.
  - Un firmware pour la stack BLE: celui-ci est déjà présent, vous n'avez pas besoin d'y toucher.
  - Un système de fichiers pour accueillir les scripts MicroPython: **c'est ici votre zone de jeu :-)**
    - Note: le **dongle USB** contient déjà les fichiers MicroPython pour discuter avec le PC et avec le robot en Bluetooth BLE ➔ Vous n'avez donc pas besoin de modifier ce code contenu dans le dongle USB.
- En cas de problème, **n'hésitez pas à contacter l'équipe ST** qui pourra remettre à jour le contenu du dongle USB et/ou effacer de manière propre le contenu du système de fichiers de vos cartes…
- Pour les amateurs d'électronique: plus d'informations, dont les schémas sur le site WaveShare.

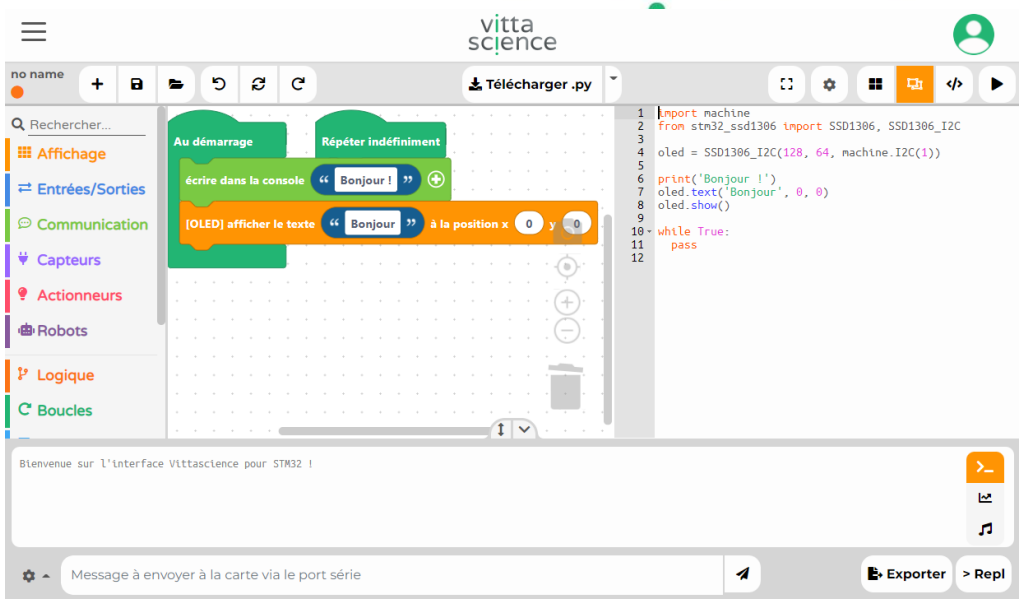
MicroPython useful libraries

- MicroPython official web site
- Documentation
- All available library list
  - **Python standard libraries**
    - sys, os, struct… **but only a subset of standard libraries are available!**
  - **MicroPython-specific libraries**
    - mainly linked to embedded hw
    - Machine: reset, i2c, spi, adc, pwm, gpio...
    - Bluetooth Low Energy (BLE): only BLE mode is supported on STM32WB55
  - **MicroPython port-specific libraries**
    - Stm32 mcu

VittaScience

VittaScience useful libraries (including stm32 ones)

- VittaScience official web site
- VittaScience stm32 online simulator
- VittaScience stm32 github
  - Libraries for the lcd, sensors…
  - and everything for the robot !



life.augmented

L’environnement logiciel de développement du robot

- **Système**: Windows, Linux ou macOS, à vous de choisir :-)
- **Editeur Python**: Votre éditeur de code favoris (Visual Studio Code, GVim, Emacs, NotePad++, Sublime Text…). Pour l’éducation, nous utilisons souvent l’éditeur très simple Thonny…
- **Terminal de communication**: votre terminal de communication préféré (minicom, hyperterminal, PuTTy, TeraTerm, pyterm, gtkterm…) avec la configuration **115200-8-N-1**: Windows (COMx) , Linux (/dev/ttyACMx).
- **Tips**: Dans la console,
  - **CTRL+D** pour redémarrer la carte
  - **CTRL+C** pour arrêter le programme MicroPython en cours.
- N’hésitez pas à consulter le site https://stm32python.gitlab.io/fr/ qui contient quelques informations complémentaires…

life.augmented

Quelques réponses à vos questions…

- **Comment mettre mon code MicroPython sur le robot ou sur la carte supplémentaire?**
  - Suivez la procédure décrite à partir de la **page 28** du Guide d'utilisation - Robot martien version STM32 vous n'avez besoin que du mode « **5V VIN** » pour le robot et « **USB_MCU** » pour la carte supplémentaire ou plus simplement, **ne touchez pas aux cavaliers de configuration car ils sont déjà à la bonne place ;-)**
  - Dans votre explorateur de fichiers, un nouveau lecteur au nom de « **PYBFLASH** » apparait, copiez vos fichiers MicroPython dedans, **main.py** sera le 1er fichier de votre application. Quelques commandes d'exemples:
    - Linux: `$ cp main.py /media/$USER/PYBFLASH; sync`
    - macOS: `$ cp main.py /Volumes/PYBFLASH; sync`
  - **IMPORTANT pour éviter le risque de corruption du système de fichiers!**
    - Lors de la mise à jour des fichiers, une LED rouge (LED3) s'allume; attendez bien qu'elle s'éteigne avant de faire le reset ou de retirer le câble USB…
    - Si vous avez besoin de retirer le câble USB (notamment pour laisser le robot autonome), n'oubliez pas avant de retirer le câble USB de "démonter" le lecteur « PYBFLASH ».
- **Comment recharger les piles du robot?** Suivez les informations **page 29** du Guide d'utilisation - Robot martien version STM32

life.augmented

Quelques réponses à vos questions…

- **Le robot est utilisé par mes collègues, comment faire pour travailler sur cette énigme en parallèle?** Plusieurs solutions:
  - Dans votre kit, il y a une carte supplémentaire Nucleo STM32WB55 et vous pouvez donc travailler dessus en MicroPython.
  - Vous pouvez aussi développer votre code en Python sur votre PC et vous ferez le portage plus tard. Attention de ne pas utiliser de dépendance à des bibliothèques Python non disponibles en MicroPython.